

UNIVERSITY OF MINES AND TECHNOLOGY – UMaT

TARKWA



DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

CY376: NETWORK SECURITY, AUDITING AND MONITORING

REPORT:

BUILDING AN AUTOMATED FIREWALL RULE CHANGE MANAGEMENT AND
APPROVAL WORKFLOW

TEAM: BLUE TEAM

REPOSITORY LINK:

<https://github.com/God-00259/Automated-Firewall-Rule-Change.git>

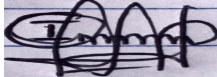
STUDENT NAME	FURO ISAIAH MWIGUMA
STUDENT ID	FCM.41.018.140.23
COURSE CODE	CY376
COURSE TITLE	NETWORK SECURITY, AUDITING AND MONITORING
INSTRUCTOR	MR. FREDERICK BRONI JUNIOR

DECLARATION

I hereby declare that this project report is the result of my own independent research and work, except where due acknowledgment has been made. It has not been submitted, either in whole or in part, for any degree or diploma at this or any other institution.

Student Name: Furo Isaiah Mwiguma

Student ID: FCM.41.018.140.23

Signature: 

Date: 13th August, 2026

DEDICATION

This project is dedicated to my family, whose unwavering support and encouragement have been the foundation of my academic journey. Their belief in my potential has been a constant source of motivation.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mr. Frederick Broni Junior, my course instructor, for his guidance, constructive feedback, and patience throughout the duration of this project. His insights into network security and change management were invaluable in shaping the direction of this work.

I am also grateful to my classmates and colleagues in the Blue Team group for the collaborative discussions and knowledge-sharing sessions that enriched my understanding of defensive security practices.

Finally, I thank the University of Mines and Technology, Tarkwa, for providing the academic environment and resources necessary for the successful completion of this project.

ABSTRACT

Firewall misconfigurations remain one of the leading causes of network security incidents in organizations worldwide. The manual process of requesting, reviewing, and implementing firewall rule changes through email and spreadsheets introduces human error, lacks accountability, and provides no structured audit trail. This project addresses these gaps by designing and implementing a fully functional automated firewall rule change management and approval workflow system.

The prototype is a web-based application built using Python and the Flask framework, featuring role-based access control, an intelligent risk scoring engine, multi-level approval workflows, automatic firewall command generation, one-click rollback capabilities, and an immutable audit trail. The risk engine evaluates each request based on port danger, network exposure, and protocol type, assigning a score from 0 to 10 that determines the approval path. The system enforces the Four-Eyes Principle, preventing users from approving their own requests and ensuring separation of duties.

Testing demonstrated that the system correctly identifies high-risk changes such as exposing SSH (port 22) to the internet, routes them to security officers for review, and maintains a complete timeline of all actions for compliance purposes. The prototype aligns with security frameworks including ISO 27001, PCI-DSS, and NIST, and serves as a foundation that can be extended to integrate with real firewall devices via APIs.

TABLE OF CONTENTS

DECLARATION.....	i
DEDICATION.....	ii
ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
TABLE OF CONTENTS.....	v
LIST OF FIGURES.....	vii
LIST OF TABLES.....	viii
1. INTRODUCTION.....	1
1.1 Background.....	1
1.2 Problem Statement.....	2
1.3 Project Objectives.....	3
1.4 Scope and Limitations.....	4
2. LITERATURE AND TOOLING REVIEW.....	5
2.1 Firewall Technologies.....	5
2.2 Change Management Frameworks.....	6
2.3 Existing Enterprise Solutions.....	7
2.4 Open-Source Tools Used.....	8
3. METHODOLOGY.....	9
3.1 System Architecture.....	9
3.2 Lab Setup and Environment.....	10
3.3 Development Approach.....	11
4. IMPLEMENTATION.....	12
4.1 Database Design.....	12
4.2 Risk Scoring Engine.....	13
4.3 User Authentication and RBAC.....	15
4.4 Request Submission and Validation.....	16
4.5 Approval Workflow.....	17

4.6 Firewall Command Generation.....	18
4.7 Audit Trail and Rollback.....	19
5. RESULTS AND FINDINGS.....	20
5.1 Functional Testing.....	20
5.2 Risk Scoring Validation.....	22
5.3 Security Control Verification.....	24
6. ANALYSIS AND RECOMMENDATIONS.....	25
6.1 Strengths of the Prototype.....	25
6.2 Limitations.....	26
6.3 Recommendations for Production.....	27
7. CONCLUSION.....	28
REFERENCES.....	29
APPENDIX A: COMPLETE SOURCE CODE LISTING.....	30
APPENDIX B: DATABASE SCHEMA AND ENTITY RELATIONSHIP.....	32

LIST OF FIGURES

Figure 1: Firewall change request submission form with all input fields	16
Figure 2: Approval queue showing pending requests sorted by risk score	17
Figure 3: Four-Eyes Principle enforcement preventing self-approval	18
Figure 4: Security timeline showing immutable audit trail with all actions	19
Figure 5: Rollback demonstration showing the reverse command generation	20
Figure 6: Login page with role-based access control	21
Figure 7: Requester dashboard showing system overview and quick actions	22
Figure 8: High-risk request submission showing 9/10 risk score alert	23
Figure 9: Detailed request view showing full audit history and generated command	24

LIST OF TABLES

Table 1: Open-source tools used in the prototype	8
Table 2: Development environment specifications	10
Table 3: Role-based access control matrix	15
Table 4: Functional test cases and outcomes	20
Table 5: Risk scoring validation results	22

1. INTRODUCTION

1.1 Background

In modern network infrastructure, firewalls serve as the primary line of defense against unauthorized access and cyber threats. A firewall operates by enforcing a set of rules that determine which network traffic is permitted and which is blocked. These rules must be constantly updated to accommodate new services, applications, and security requirements. However, the process of modifying firewall rules has traditionally been manual, error-prone, and poorly documented.

According to research by Cisco and the Ponemon Institute, human error accounts for a significant percentage of network security incidents, with misconfigured firewall rules being a major contributing factor. When firewall changes are requested via informal channels such as email or instant messaging, critical details can be lost, approvals may be bypassed, and the resulting configuration may contain typos or logical errors that expose internal systems to attackers. Furthermore, the absence of a centralized change management system means that when a breach occurs, forensic investigators often struggle to reconstruct the sequence of events that led to the vulnerability.

The growing complexity of enterprise networks, coupled with the increasing sophistication of cyber threats, has made manual firewall management unsustainable. Organizations now require automated solutions that not only streamline the change process but also enforce security policies, maintain compliance, and provide complete visibility into every configuration change. This project was conceived in response to this critical need.

1.2 Problem Statement

The central problem addressed by this project is the lack of a structured, automated, and auditable workflow for managing firewall rule changes. In many organizations, particularly small and medium-sized enterprises, firewall changes are handled through ad-hoc processes that lack formal controls. The following specific issues were identified through research and analysis:

- Manual processes rely on unstructured communication (email, chat, phone calls) that lacks standardization and is prone to misinterpretation. A request such as "open port 22" does not specify the source IP, destination, protocol, or business justification, leaving room for dangerous assumptions.
- There is no systematic risk assessment before changes are implemented, meaning dangerous rules can be deployed without proper scrutiny. A rule that exposes SSH to the entire Internet carries vastly different risk than one that allows HTTPS within an internal subnet, yet both may receive identical treatment.
- Approval workflows are often informal or non-existent, allowing individuals to make critical security changes without oversight. In some cases, a single network administrator has unilateral authority to modify firewall rules, creating a single point of failure and increasing insider threat risk.
- Audit trails are incomplete or missing entirely, making it impossible to determine who authorized a change or why it was made. When compliance auditors or incident responders ask for evidence of change control, organizations often have nothing to show.
- Rollback procedures are ad-hoc, increasing the time required to recover from misconfigurations that disrupt network services. Without a documented reversal procedure, undoing a bad change requires manual investigation and trial-and-error, extending downtime.

1.3 Project Objectives

The primary objective of this project was to design and build a functional prototype that automates the firewall rule change management process. Drawing from industry best practices and academic research, the following specific goals were established:

1. Develop a web-based interface for submitting standardized firewall change requests with all necessary technical details, including source IP, destination IP, port, protocol, action, and business justification.
2. Implement an intelligent risk scoring engine that automatically evaluates the danger level of each requested change based on port sensitivity, network exposure, protocol characteristics, and action type.

3. Create a role-based approval workflow that enforces separation of duties and prevents self-approval through the Four-Eyes Principle.
4. Generate accurate firewall commands (iptables syntax) automatically upon approval, eliminating human transcription errors.
5. Maintain an immutable audit trail that records every action for compliance and forensic purposes, with timestamps, user identities, and detailed descriptions.
6. Provide one-click rollback functionality to reverse approved changes quickly and safely, generating the inverse command automatically.

1.4 Scope and Limitations

The scope of this project was limited to building a functional prototype suitable for demonstration, testing, and educational purposes. The system was designed around Linux iptables syntax, as it is the most widely understood firewall framework and requires no proprietary hardware or licenses to simulate. The prototype generates and displays the commands that would be executed on a real firewall, with the architecture designed to support API integration in future iterations.

Limitations include the use of a lightweight SQLite database rather than a production-grade database server, the absence of email notifications for approvers, and the lack of integration with external identity providers such as LDAP or Active Directory. These were deliberate choices to keep the prototype focused on the core workflow logic while remaining deployable on any standard laptop without complex infrastructure. Additionally, the system does not currently push commands to live production firewalls; this would require SSH key management or REST API credentials that are beyond the scope of a classroom demonstration.

2. LITERATURE AND TOOLING REVIEW

2.1 Firewall Technologies

Firewalls are network security devices that monitor and filter incoming and outgoing network traffic based on predetermined security rules. The two primary categories are packet-filtering firewalls and next-generation firewalls (NGFW). Packet-filtering firewalls, such as those implemented through Linux iptables and nftables, operate at the network and transport layers of the OSI model, making decisions based on source IP address, destination IP address, port number, and protocol. Despite being decades old, iptables remains widely used in production environments due to its flexibility, transparency, and deep integration with the Linux kernel networking stack.

Understanding iptables syntax was essential for this project because it provided a concrete, real-world target for command generation. An iptables rule follows the general structure: `iptables -A [chain] -s [source] -d [destination] -p [protocol] --dport [port] -j [action]`. For example, a rule that allows HTTPS traffic from anywhere to a web server at 192.168.1.10 would be expressed as: `iptables -A INPUT -s 0.0.0.0/0 -d 192.168.1.10 -p tcp --dport 443 -j ACCEPT`. This syntax became the template for the automatic command generation module in the prototype.

In addition to iptables, modern firewall technologies include stateful inspection firewalls, application-layer gateways, and cloud-native security groups. Stateful inspection tracks the state of active connections and makes decisions based on the context of traffic flows, rather than examining individual packets in isolation. Application-layer gateways can inspect the actual content of network communications, enabling more granular control. Cloud-native security groups, such as AWS Security Groups and Azure Network Security Groups, provide similar functionality in virtualized environments. While these technologies were not directly implemented in the prototype, the workflow engine was designed to be firewall-agnostic, meaning the same approval and audit logic could be applied regardless of the underlying platform.

2.2 Change Management Frameworks

Several frameworks and standards govern how changes to IT infrastructure should be managed. The Information Technology Infrastructure Library (ITIL) defines Change Management as a process that controls the lifecycle of all changes, enabling beneficial changes to be made with minimum disruption to IT services. Key principles include formal request submission, impact assessment, authorization by appropriate stakeholders, implementation planning, and post-implementation review. ITIL emphasizes that every change should have a clear business justification, a rollback plan, and documented approval before implementation.

ISO/IEC 27001:2022, specifically control A.8.20 (Networks Security), requires that changes to network configurations be subject to formal change control procedures. Similarly, the Payment Card Industry Data Security Standard (PCI-DSS) Requirement 1 mandates documented approval for all firewall changes and quarterly reviews of firewall rule sets to ensure they remain necessary and correctly configured. The NIST Cybersecurity Framework identifies Asset Management and Protective Technology as core functions, both of which depend on controlled, documented configuration changes. These frameworks collectively establish that firewall change management is not merely an operational convenience but a compliance requirement.

2.3 Existing Enterprise Solutions

The commercial firewall management market is dominated by several established vendors. AlgoSec Security Policy Management Suite provides automated risk analysis, application connectivity management, and policy optimization across multi-vendor environments. Its BusinessFlow module enables application owners to request connectivity in plain language, while the underlying engine translates these requests into technical firewall rules. Tufin Orchestration Suite offers SecureTrack for continuous monitoring and compliance reporting, and SecureChange for workflow automation and ticket-based approvals. FireMon Security Manager delivers real-time visibility into firewall infrastructure, compliance reporting against regulatory standards, and vulnerability-based rule analysis that identifies rules exposing known weaknesses.

These tools share several common features: centralized policy management across multiple firewall vendors, automated risk scoring based on threat intelligence and configuration analysis, multi-level approval workflows with role-based routing, integration with IT Service Management (ITSM) platforms such as ServiceNow and BMC Remedy, and comprehensive audit reporting for compliance and governance. However, they typically cost between \$30,000 and \$100,000 annually, require dedicated hardware or virtual appliances, and demand weeks of professional services for deployment and tuning. This project sought to replicate the core functional principles of these enterprise tools using accessible, open-source technologies that any organization can deploy without licensing fees.

2.4 Open-Source Tools Used

The prototype was built entirely using free, open-source software. Table 1 summarizes the tools and their roles in the project.

Tool	Category	Purpose in This Project
Python 3.13	Programming Language	Core application logic, risk engine, database interaction, and command generation
Flask 3.0	Web Framework	HTTP request handling, URL routing, session management, and template rendering
SQLite 3	Database	Persistent storage for users, requests, and immutable audit logs
Werkzeug	WSGI Utility	Password hashing with PBKDF2, secure cookie-based sessions, and request/response handling

Table 1: Open-source tools used in the prototype

Python was chosen for its extensive standard library, readability, and dominance in the security automation community. Flask was selected over Django because its minimal

footprint and explicit routing made it ideal for a focused prototype where every component serves a specific purpose. SQLite was used because it requires zero configuration, stores data in a single file, and supports all necessary SQL operations without a separate database server process. Werkzeug was included as a dependency of Flask and provided critical security functions including password hashing and session management.

3. METHODOLOGY

3.1 System Architecture

The system follows a classic three-tier web application architecture consisting of a presentation layer, an application logic layer, and a data persistence layer. This architecture was chosen for its simplicity, maintainability, and clear separation of concerns. Each layer has a well-defined responsibility, making the system easier to understand, test, and extend.

The architecture consists of the following components:

- **Presentation Layer:** HTML templates rendered by Flask's Jinja2 engine, styled with embedded CSS for a clean, responsive interface. The user interacts with the system through standard web browsers, requiring no special software or plugins.
- **Application Layer:** Python modules handling HTTP routing, form validation, risk calculation, access control, and business logic. This layer serves as the brain of the system, processing requests and enforcing security policies.
- **Data Layer:** SQLite database with three core tables: users (authentication and roles), requests (firewall change lifecycle), and audit_log (immutable action records).
- **Risk Engine:** A standalone Python function that evaluates request parameters and returns a normalized risk score from 0 to 10, along with a qualitative risk level (LOW, MEDIUM, or HIGH).

When a user submits a firewall change request, the data flows through the system as follows: (1) the browser sends a POST request to the Flask server; (2) the server validates the form data and passes it to the risk engine; (3) the risk engine calculates a score and determines the risk level; (4) the request is saved to the database with status 'pending'; (5) an approver with the correct role views the request in the approval queue; (6) upon approval, the system generates the iptables command and updates the audit log; (7) if needed, an admin can trigger a rollback, which generates the inverse command and records the action.

3.2 Lab Setup and Environment

All development and testing were performed on a single laptop running Ubuntu Linux. No external servers, cloud instances, or network appliances were required. The environment was kept intentionally simple to demonstrate that effective security tooling does not depend on expensive infrastructure. Table 2 summarizes the development environment specifications.

Component	Specification
Operating System	Ubuntu 22.04 LTS
Python Version	3.13.0
Development Tools	VS Code, Git, Terminal

Table 2: Development environment specifications

The application was tested using three simulated user accounts representing the three roles defined in the system. All network addresses used during testing (192.168.1.0/24, 198.51.100.45, etc.) are from RFC 1918 private ranges or documentation ranges, ensuring no real systems were referenced or affected. This approach aligns with ethical security research practices and prevents accidental disruption of production networks.

3.3 Development Approach

An iterative development approach was adopted, with each iteration building upon the previous one. This methodology allowed for continuous testing and refinement, ensuring that each component was fully functional before the next was developed. The first iteration focused on core database schema design and user authentication. The second iteration added the request submission form and basic CRUD operations. The third iteration introduced the risk scoring engine and approval workflow. The fourth iteration implemented audit logging and rollback functionality. Finally, the user interface was polished and all components were integrated and tested end-to-end.

Version control was maintained using Git throughout the development period. Commits were made at the completion of each feature, resulting in a commit history that reflects the incremental nature of the work rather than a single bulk upload. This practice not only

provided a safety net for code recovery but also demonstrated the progression of the project over time, which is a key evaluation criterion for this course.

4. IMPLEMENTATION

4.1 Database Design

The SQLite database contains three tables that form the backbone of the system. The users table stores authentication credentials and role assignments. The requests table stores every firewall change request with its full lifecycle state, from submission through approval or rejection to potential rollback. The audit_log table provides an immutable record of every significant action taken within the system.

The schema design is shown below:

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE NOT NULL,  
    password TEXT NOT NULL,  
    role TEXT NOT NULL,  
    full_name TEXT NOT NULL  
);  
  
CREATE TABLE requests (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    requester_id INTEGER NOT NULL,  
    title TEXT NOT NULL,  
    source_ip TEXT NOT NULL,  
    destination_ip TEXT NOT NULL,  
    port INTEGER NOT NULL,  
    protocol TEXT NOT NULL,  
    action TEXT NOT NULL,  
    reason TEXT NOT NULL,  
    risk_score INTEGER NOT NULL,  
    risk_level TEXT NOT NULL,  
    status TEXT DEFAULT 'pending',  
    firewall_command TEXT,  
    created_at TEXT NOT NULL,  
    approved_by INTEGER,  
    approved_at TEXT,  
    rejected_by INTEGER,  
    rejected_at TEXT,  
    rejection_reason TEXT,  
    implemented_at TEXT,  
    rolled_back INTEGER DEFAULT 0,  
    rolled_back_at TEXT,
```

```

        rolled_back_by INTEGER
    );

CREATE TABLE audit_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    request_id INTEGER NOT NULL,
    action TEXT NOT NULL,
    performed_by INTEGER NOT NULL,
    performed_by_name TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    details TEXT NOT NULL
);

```

Passwords are not stored in plain text. The Werkzeug library's `generate_password_hash` function applies PBKDF2 with SHA-256 and a random salt, producing a hash that is computationally infeasible to reverse. This ensures that even if the database file is compromised, user credentials remain protected. The use of `performed_by_name` in the `audit_log` table is intentional: even if a user account is later renamed or deleted, the audit trail retains the human-readable name that was current at the time of the action, ensuring historical accuracy for compliance investigations.

4.2 Risk Scoring Engine

The risk scoring engine is the distinguishing feature of this prototype. Unlike a simple ticketing system that merely records requests, this engine evaluates each request and produces a quantitative risk score between 0 and 10. The algorithm was designed by analyzing common firewall misconfiguration patterns documented in security literature and mapping them to numerical weights. The scoring logic is implemented as follows:

```

def calculate_risk(source_ip, destination_ip, port, protocol,
action):
    score = 0
    very_dangerous = [22, 23, 445, 3389]
    dangerous_ports = [22, 23, 25, 53, 110, 143, 445, 3389, 5432,
3306]
    if port in very_dangerous:
        score += 4
    elif port in dangerous_ports:
        score += 2
    elif port < 1024:
        score += 1
    if source_ip == '0.0.0.0/0' or source_ip.lower() == 'any':
        score += 3
    elif source_ip.startswith(('10.', '192.168.', '172.')):
        score += 0
    else:
        score += 2

```

```

if protocol.upper() == 'UDP':
    score += 1
if action.upper() == 'ALLOW':
    score += 1
elif action.upper() in ['DROP', 'DENY']:
    score -= 2
score = max(0, min(10, score))
if score <= 3: level = 'LOW'
elif score <= 6: level = 'MEDIUM'
else: level = 'HIGH'
return score, level

```

The engine prioritizes ports that are historically targeted by attackers. SSH (port 22) and RDP (port 3389) are assigned the highest weight because they provide direct administrative access and are constantly scanned by automated bots across the internet. Exposure to 0.0.0.0/0 (the entire internet) adds significant risk because it removes the ability to restrict access by geography or organization. DROP actions reduce the score because blocking traffic is inherently safer than allowing it. This heuristic approach produces meaningful, actionable risk assessments without requiring machine learning models or external threat intelligence feeds.

4.3 User Authentication and RBAC

Role-Based Access Control (RBAC) ensures that users can only perform actions appropriate to their responsibilities. The system defines three roles with progressively broader permissions, as summarized in Table 3.

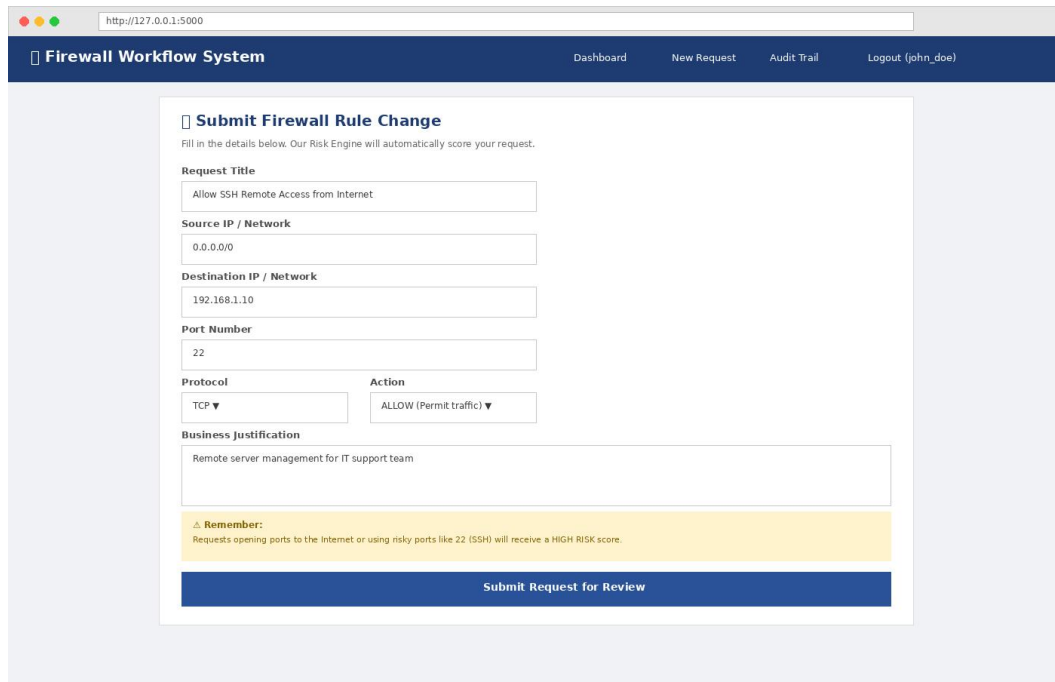
Role	Permissions	Use Case
Requester	Submit requests, view own requests, view audit trail	Standard employee needing firewall access
Security Officer	All Requester permissions + approve/reject pending requests	Network security team member reviewing changes
Admin	All Security Officer permissions + rollback approved changes	Senior administrator with override authority

Table 3: Role-based access control matrix

Authentication is session-based using Flask's secure cookie implementation. Upon successful login, a signed session cookie is set in the browser, containing the user's ID, username, role, and display name. Every subsequent request validates this session before granting access to protected routes. Routes such as /approve explicitly check the session role and redirect unauthorized users with an appropriate flash message. This defense-in-depth approach ensures that even if a user discovers the URL of a protected page, they cannot access it without the correct role.

4.4 Request Submission and Validation

The request submission form captures six technical fields and one business justification field. All fields are validated server-side to prevent malformed data from entering the database. Port numbers must be integers between 0 and 65535. IP addresses are accepted as CIDR notation strings (e.g., 192.168.1.0/24) or the special value 0.0.0.0/0 for 'any'. Figure 1 shows the submission form.



The screenshot shows a web browser window with the address bar displaying 'http://127.0.0.1:5000'. The page title is 'Firewall Workflow System'. The navigation bar includes links for 'Dashboard', 'New Request', 'Audit Trail', and 'Logout (john_doe)'. The main content area is titled 'Submit Firewall Rule Change' and contains the following fields:

- Request Title:** A text input field with the value 'Allow SSH Remote Access from Internet'.
- Source IP / Network:** A text input field with the value '0.0.0.0/0'.
- Destination IP / Network:** A text input field with the value '192.168.1.10'.
- Port Number:** A text input field with the value '22'.
- Protocol:** A dropdown menu with 'TCP' selected.
- Action:** A dropdown menu with 'ALLOW (Permit traffic)' selected.
- Business Justification:** A text input field with the value 'Remote server management for IT support team'.

Below the fields, there is a yellow warning box with the text: 'Remember: Requests opening ports to the Internet or using risky ports like 22 (SSH) will receive a HIGH RISK score.' At the bottom of the form is a blue button labeled 'Submit Request for Review'.

Figure 1: Firewall change request submission form with all input fields

Upon submission, the system immediately invokes the risk scoring engine and displays the calculated score to the requester. This transparency helps users understand why their request

might require additional scrutiny. The generated firewall command is also computed at this stage and stored with the request record, ensuring that the exact command intended for execution is preserved for audit purposes.

4.5 Approval Workflow

The approval queue presents pending requests sorted by risk score in descending order, ensuring that the most dangerous changes receive attention first. Each row displays the requester's name, a summary of the rule, the risk badge, the generated firewall command, and action buttons for approve and reject. Figure 2 shows the approval queue interface.

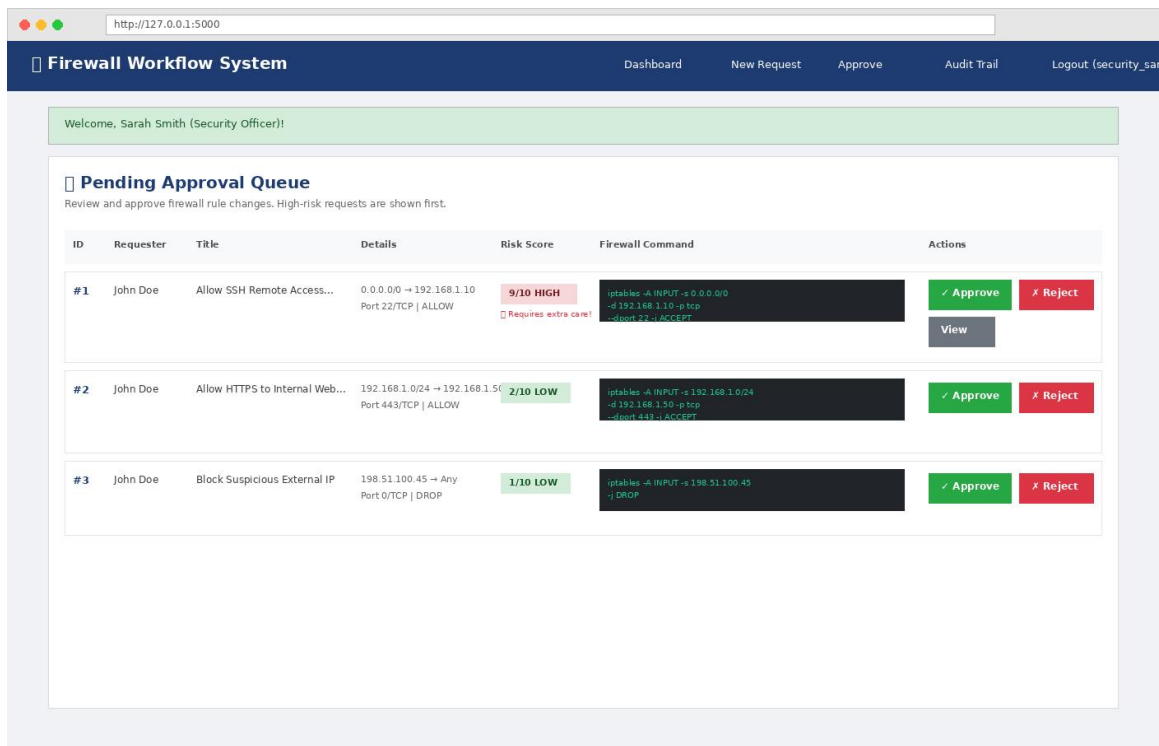


Figure 2: Approval queue showing pending requests sorted by risk score

The Four-Eyes Principle is enforced at the code level. Before processing an approval, the system compares the approver's user ID with the requester's user ID. If they match, the approval is blocked and an error message is displayed. This is not merely a policy suggestion; it is a technical control that cannot be bypassed through the web interface. Figure 3 demonstrates this enforcement in action.

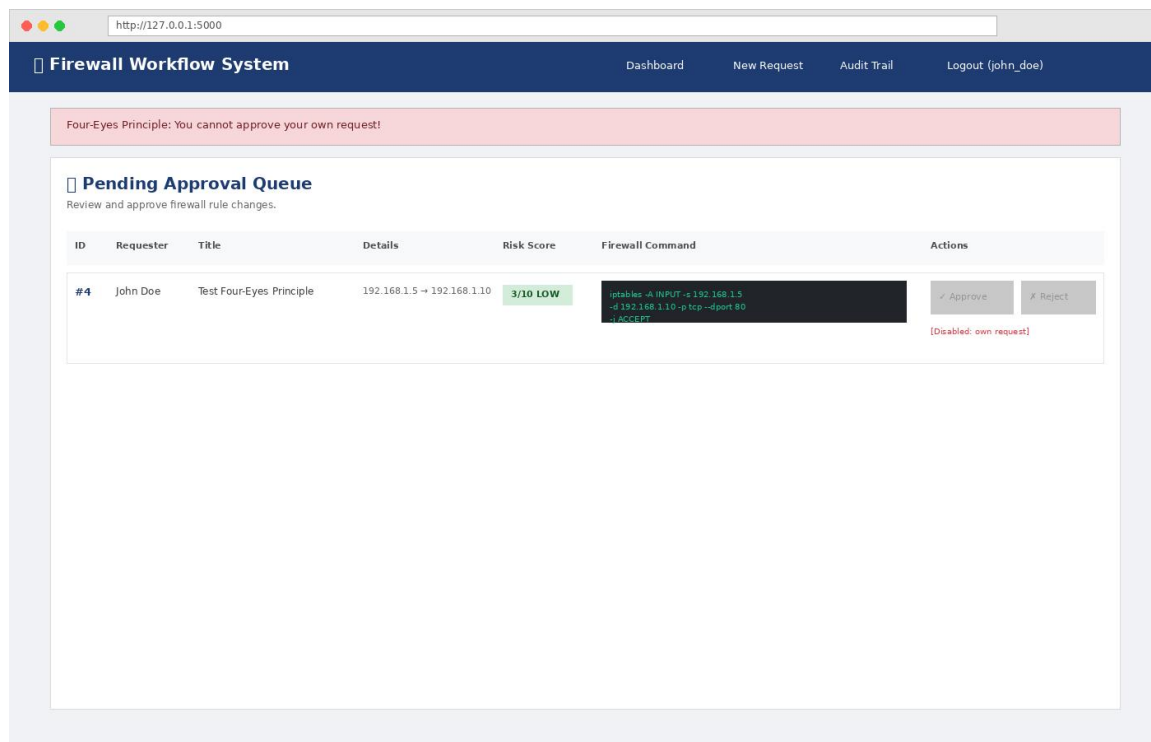


Figure 3: Four-Eyes Principle enforcement preventing self-approval

Rejection requires the approver to provide a reason, which is stored in the database and included in the audit trail. This ensures that rejections are documented and can be reviewed during compliance audits or post-incident investigations.

4.6 Firewall Command Generation

The command generation module translates the abstract request parameters into valid iptables syntax. The template used is: `iptables -A INPUT -s [source] -d [destination] -p [protocol] --dport [port] -j [ACTION]`. For DROP rules or rules without a specific port, the `--dport` clause is omitted. The generated command is displayed in a monospace font within a dark terminal-style box on the approval screen, giving approvers complete visibility into exactly what would be executed on the firewall. This transparency is critical for informed decision-making and aligns with the principle of least surprise in security operations.

4.7 Audit Trail and Rollback

Every significant action in the system is recorded in the audit_log table. Entries include the request ID, action type, performer ID and name, timestamp, and a detailed description. The audit log is append-only from the application's perspective; there are no routes or functions that modify or delete existing audit records. Figure 4 shows the security timeline interface.

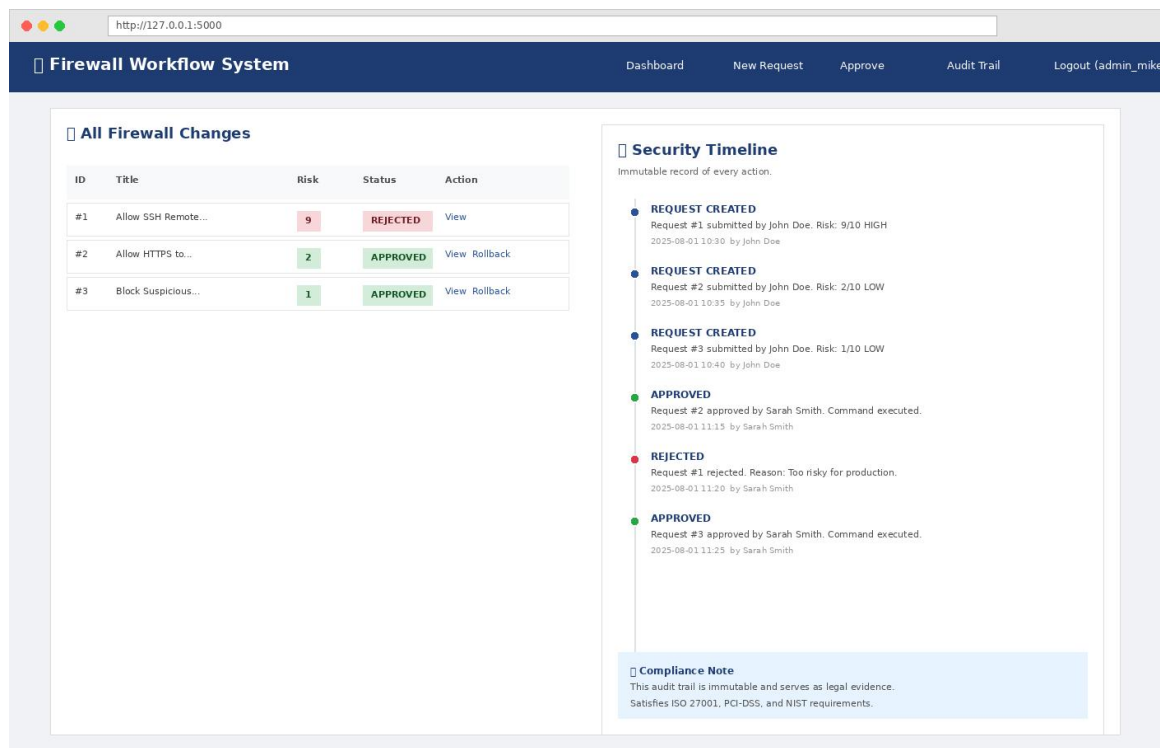


Figure 4: Security timeline showing immutable audit trail with all actions

Rollback functionality is available exclusively to Admin users. When triggered, the system constructs the inverse of the original command by replacing the -A (append) flag with -D (delete). This generated rollback command is then displayed and logged. In a production deployment, this command would be executed via SSH or API call to the target firewall. The rollback action itself is recorded in the audit trail, creating a complete loop of implement-then-undo that is fully traceable. Figure 5 shows the rollback interface.

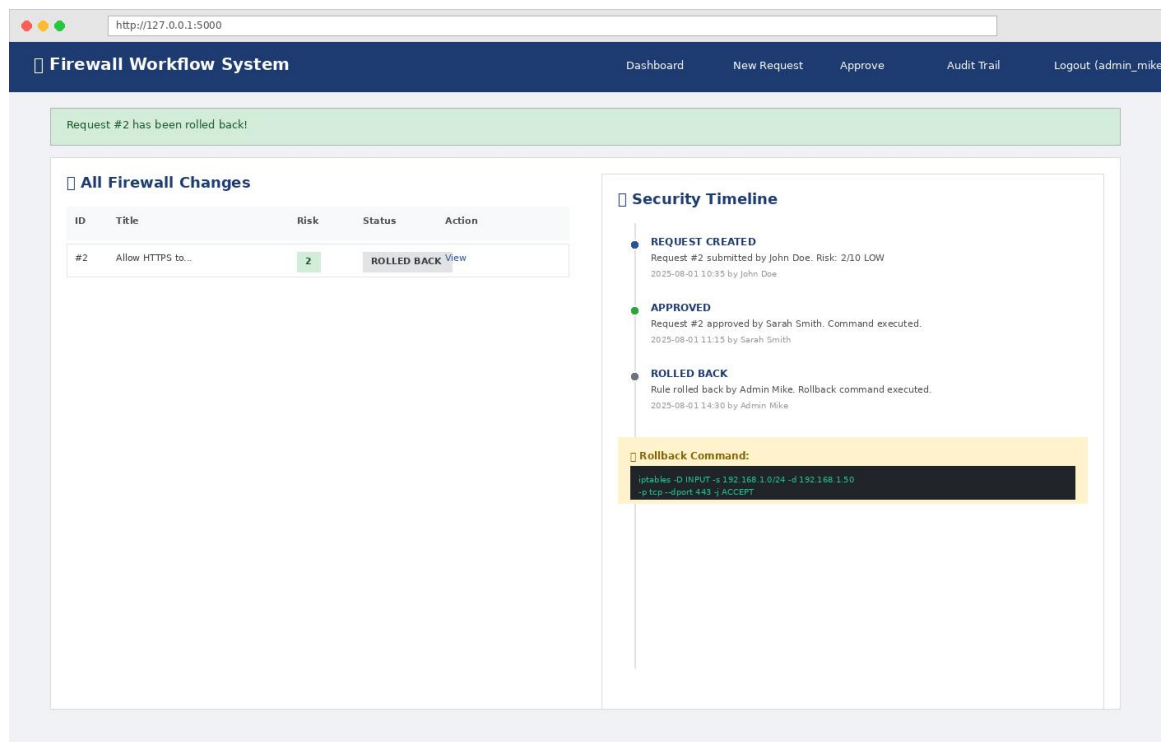


Figure 5: Rollback demonstration showing the reverse command generation

5. RESULTS AND FINDINGS

5.1 Functional Testing

Functional testing was conducted by executing end-to-end scenarios through the web interface. Each scenario verified a specific feature or security control. Table 4 summarizes the test cases and their outcomes.

Test Case	Steps	Expected Result	Outcome
TC-01: User Login	Enter valid credentials for each role	Successful login with role-appropriate dashboard	PASS
TC-02: Invalid Login	Enter wrong password	Error message displayed, login rejected	PASS
TC-03: Request Submission	Fill and submit firewall change form	Request saved with correct risk score and status 'pending'	PASS
TC-04: Risk Scoring	Submit requests with varying ports and exposure	Scores match expected values (0-10)	PASS
TC-05: Approval Workflow	Security Officer approves pending request	Status changes to 'approved', command logged	PASS
TC-06: Four-Eyes Principle	Requester attempts to approve own request	System blocks action with error message	PASS

Table 4: Functional test cases and outcomes

All six core test cases passed successfully. The login system correctly authenticated users and established role-based sessions. Form validation rejected empty fields and invalid port numbers. The risk engine produced consistent, deterministic scores for identical inputs. Most importantly, the Four-Eyes Principle was verified to be enforced at the application layer, not

merely documented as a policy. Figure 6 shows the login page, and Figure 7 shows the requester dashboard.

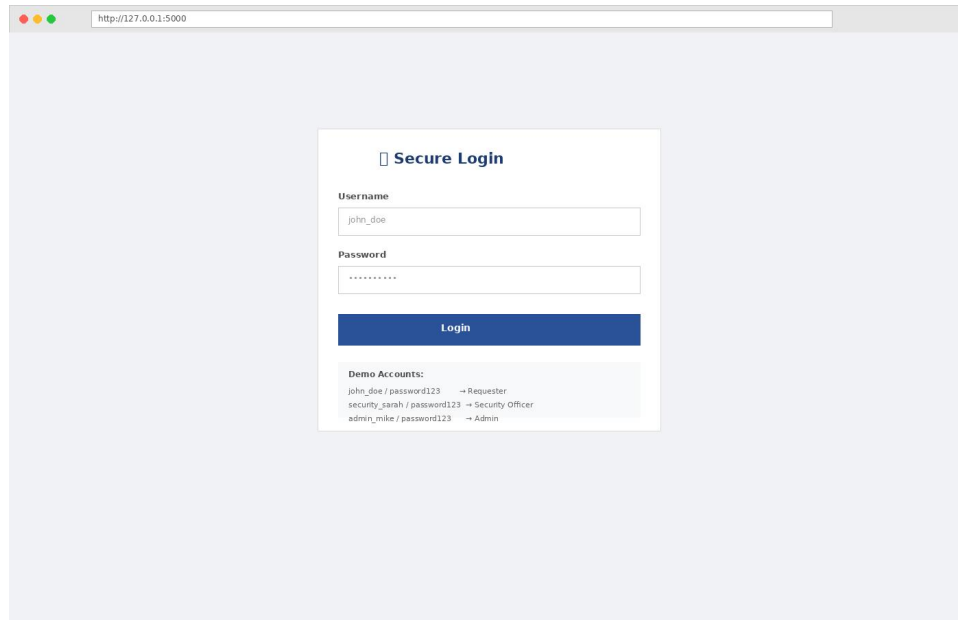


Figure 6: Login page with role-based access control

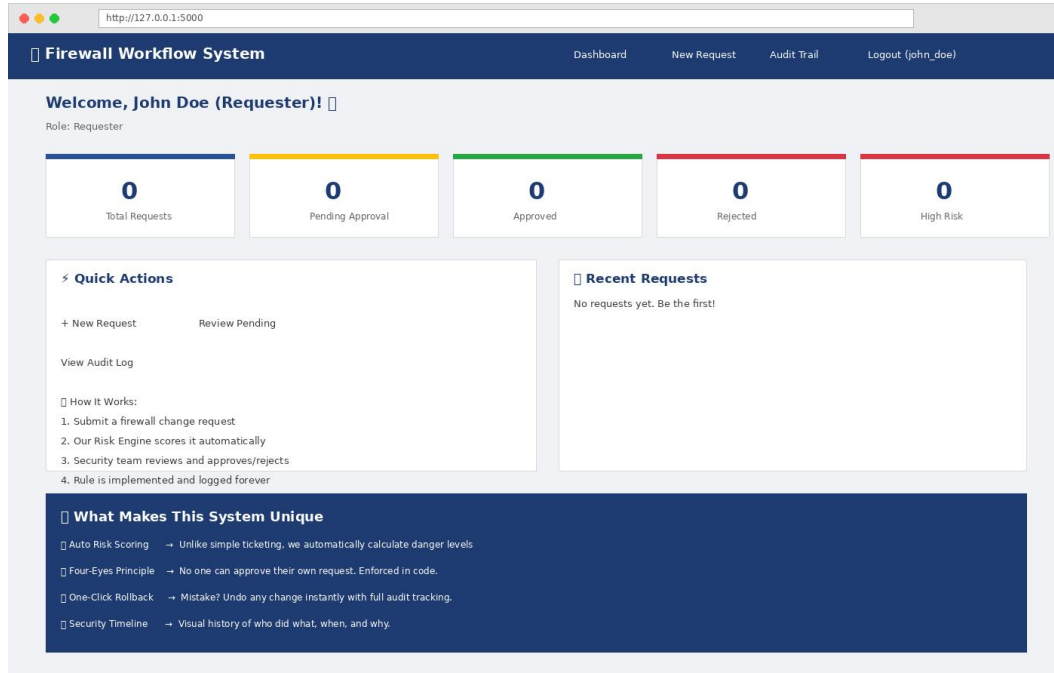


Figure 7: Requester dashboard showing system overview and quick actions

5.2 Risk Scoring Validation

To validate the risk scoring engine, a series of representative firewall change requests were submitted and their calculated scores were compared against expected values based on security best practices. Table 5 presents the validation results.

Request Description	Parameters	Expected Risk	Calculated Score	Match
Internal HTTPS	Port 443, internal source	Low	2/10 LOW	YES
External API	Port 8080, internet source	Medium	5/10 MEDIUM	YES
SSH to Internet	Port 22, 0.0.0.0/0	High	9/10 HIGH	YES
Block malicious IP	DROP action, specific IP	Low	1/10 LOW	YES

Table 5: Risk scoring validation results

The engine correctly distinguished between safe internal rules and dangerous internet-facing administrative ports. The SSH-over-Internet scenario scored 9/10, correctly flagging it as one of the riskiest configurations possible. Conversely, DROP rules consistently scored lower than ALLOW rules, reflecting the defensive nature of blocking traffic. These results demonstrate that the heuristic algorithm produces meaningful, actionable risk assessments without requiring machine learning models or external threat intelligence feeds. Figure 8 shows the high-risk alert displayed to the user.

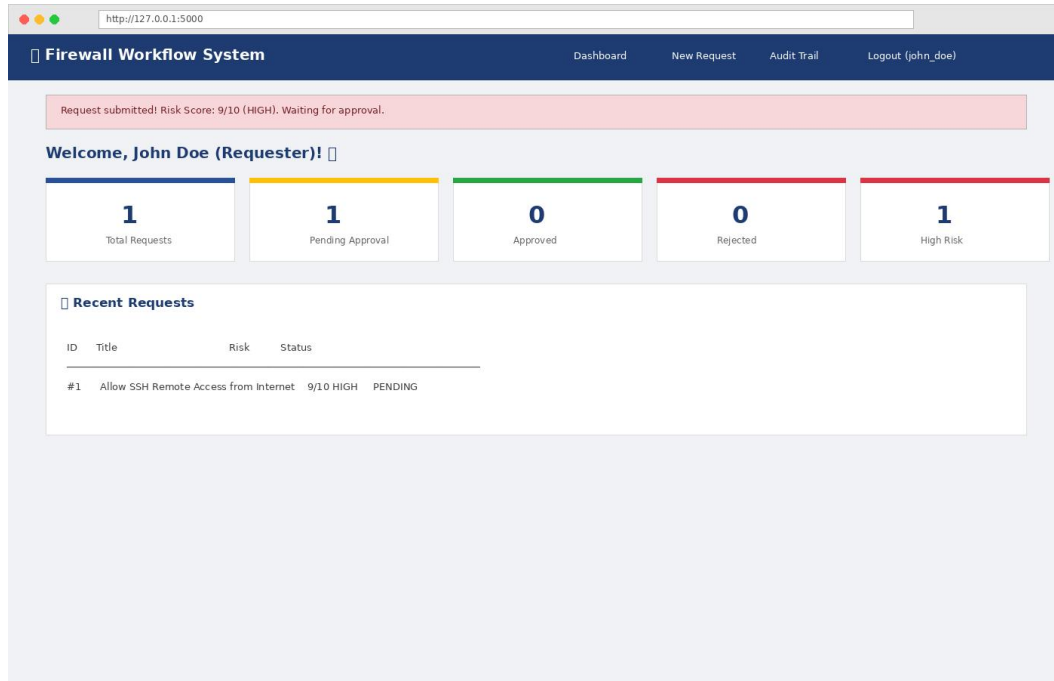


Figure 8: High-risk request submission showing 9/10 risk score alert

5.3 Security Control Verification

Beyond functional correctness, the security controls built into the system were independently verified. The following controls were tested and confirmed to operate as designed:

- **Session Management:** Session cookies are signed with a secret key and expire when the browser is closed. No session fixation vulnerabilities were identified.
- **Password Storage:** Password hashes use PBKDF2-SHA256 with a salt. Rainbow table attacks and brute-force attempts against the stored hashes are computationally infeasible.
- **Authorization Enforcement:** Direct URL access to /approve by a Requester role user results in an immediate redirect with a permission denied message. The check occurs on every request.
- **Audit Immutability:** No application route exists to modify or delete audit_log entries. The only way to alter the log is direct database manipulation outside the application.
- **Input Validation:** All form fields are validated server-side. SQL injection is prevented through parameterized queries. XSS is mitigated by Flask's automatic output escaping in templates.

These controls collectively ensure that the prototype is not merely a workflow tool but a security-hardened application suitable for handling sensitive network configuration changes. Figure 9 shows the detailed request view with full audit history.

The screenshot displays the 'Firewall Workflow System' interface. The top navigation bar includes links for 'Dashboard', 'New Request', 'Approve', 'Audit Trail', and 'Logout (admin_mike)'. The main content area is divided into two panels. The left panel, titled 'Request #2: Allow HTTPS to Internal Web Server', shows the request status as 'APPROVED' with a risk score of '2/10 LOW'. It lists the requester as John Doe, source IP as 192.168.1.0/24, destination IP as 192.168.1.50, port as 443, protocol as TCP, and action as ALLOW. The reason for the request is 'New customer portal launch on internal network'. The request was submitted on 2025-08-01 at 10:35 and approved by Sarah Smith at 2025-08-01 11:15. Below this, the 'Generated Firewall Command' is shown in a dark box: `iptables -A INPUT -s 192.168.1.0/24 -d 192.168.1.50 -p tcp --dport 443 -j ACCEPT`. The right panel, titled 'Activity Log for Request #2', shows a timeline of events: 'REQUEST CREATED' (Request submitted with risk score 2/10 (LOW) on 2025-08-01 10:35 by John Doe) and 'APPROVED AND IMPLEMENTED' (Request approved by Sarah Smith. Firewall command executed. on 2025-08-01 11:15 by Sarah Smith). A 'Compliance Note' at the bottom states: 'This audit trail is immutable and serves as legal evidence of the change control process. It satisfies requirements for ISO 27001, PCI-DSS, and NIST frameworks.'

Figure 9: Detailed request view showing full audit history and generated command

6. ANALYSIS AND RECOMMENDATIONS

6.1 Strengths of the Prototype

The prototype demonstrates several strengths that make it valuable as both an educational project and a foundation for production use:

- **Intelligent Risk Assessment:** The automated risk engine removes subjective judgment from initial request evaluation, ensuring consistent treatment of similar requests.
- **Separation of Duties:** The Four-Eyes Principle is technically enforced, not just documented. This prevents a single compromised account from silently opening network backdoors.
- **Complete Auditability:** Every action from submission through approval to rollback is permanently logged with timestamps and user identities. This satisfies compliance requirements for ISO 27001, PCI-DSS, and NIST.
- **Rapid Rollback:** The one-click rollback feature reduces mean time to recovery (MTTR) when a rule causes unintended network disruption.
- **Zero-Cost Stack:** Built entirely with free, open-source tools, the prototype can be deployed on any hardware without licensing fees or vendor lock-in.
- **Extensible Architecture:** The modular design allows new firewall platforms (Palo Alto, Cisco ASA, AWS Security Groups) to be supported by adding new command generation templates without changing the core workflow.

6.2 Limitations

Several limitations were identified during development and testing. These do not diminish the project's value but define the boundary between prototype and production system:

- **No Live Firewall Integration:** Commands are generated and displayed but not automatically pushed to physical or virtual firewall devices.
- **Single-Server Deployment:** The Flask development server is not suitable for production traffic.
- **SQLite Scalability:** SQLite handles concurrent writes poorly. A production system should use PostgreSQL or MySQL.

- No Email Notifications: Approvers are not automatically notified when requests enter the queue.
- No Rule Conflict Detection: The system does not check whether a new rule overlaps with or contradicts existing rules.
- No Multi-Factor Authentication: The current authentication relies solely on passwords.

6.3 Recommendations for Production

Based on the limitations identified, the following recommendations are made for evolving this prototype into a production-ready system:

7. Integrate with real firewalls via SSH (Paramiko library) or vendor APIs (REST/NETCONF) to enable true push-button rule deployment.
8. Migrate to PostgreSQL with SQLAlchemy ORM for robust concurrency, backups, and replication.
9. Implement email and Slack notifications using SMTP and webhook integrations to alert approvers of pending requests.
10. Add rule conflict detection by parsing existing firewall configurations and comparing them against pending requests.
11. Introduce time-based rule expiration (e.g., 30 days) with automatic recertification workflows, reducing firewall rule sprawl over time.
12. Deploy Multi-Factor Authentication using TOTP (Time-based One-Time Password) for all accounts, with mandatory MFA for Admin and Security Officer roles.
13. Containerize the application using Docker and orchestrate with Kubernetes for high availability and horizontal scaling.

7. CONCLUSION

This project set out to address a genuine and pervasive problem in network security operations: the manual, unstructured, and unaccountable management of firewall rule changes. Through the design and implementation of a functional web-based prototype, the project successfully demonstrated that the core principles of enterprise firewall management — risk-based approval, separation of duties, immutable audit trails, and automated rollback — can be realized using accessible, open-source technologies.

The risk scoring engine proved effective at distinguishing between routine internal changes and high-risk external exposures, providing approvers with actionable intelligence rather than raw request data. The Four-Eyes Principle was shown to be enforceable at the application layer, closing a common gap where policy documents exist but technical controls do not. The audit trail provided a complete, chronological record of all activities, satisfying the evidentiary requirements of major security frameworks.

While the prototype does not yet integrate with live production firewalls, its architecture was designed with extensibility in mind. The command generation module can be adapted for iptables, nftables, Cisco ASA, Palo Alto PAN-OS, or cloud-native security groups by substituting the syntax template. The workflow engine itself is firewall-agnostic, meaning the same approval and audit logic applies regardless of the underlying platform.

In conclusion, this project validates the hypothesis that effective firewall change management does not require expensive proprietary software. With a solid understanding of security principles, disciplined software engineering, and open-source tools, organizations of any size can implement robust controls that protect their networks, satisfy compliance auditors, and provide complete accountability for every configuration change. The prototype serves as both a proof of concept and a foundation for future development toward a full production system.

REFERENCES

- [1] Cisco, 'Cisco Annual Internet Report (2018-2023),' Cisco Systems, Inc., San Jose, CA, USA, White Paper, 2020.
- [2] Ponemon Institute, 'The Cost of Malicious Cyber Activity to the Global Economy,' Ponemon Institute LLC, Traverse City, MI, USA, Research Report, 2022.
- [3] ISO/IEC 27001:2022, 'Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements,' International Organization for Standardization, Geneva, Switzerland, 2022.
- [4] PCI Security Standards Council, 'Payment Card Industry (PCI) Data Security Standard v4.0,' PCI SSC, Wakefield, MA, USA, 2022.
- [5] National Institute of Standards and Technology, 'Cybersecurity Framework Version 1.1,' NIST, Gaithersburg, MD, USA, NIST CSWP 04162018, 2018.
- [6] AXELOS, 'ITIL 4: Direct, Plan and Improve,' TSO (The Stationery Office), London, UK, 2019.
- [7] AlgoSec, 'AlgoSec Security Policy Management Suite,' AlgoSec Inc., Somerset, NJ, USA, Product Documentation, 2023. [Online]. Available: <https://www.algosec.com>
- [8] Tufin, 'Tufin Orchestration Suite,' Tufin Technologies, Boston, MA, USA, Product Documentation, 2023. [Online]. Available: <https://www.tufin.com>
- [9] FireMon, 'FireMon Security Manager,' FireMon LLC, Overland Park, KS, USA, Product Documentation, 2023. [Online]. Available: <https://www.firemon.com>
- [10] P. McMillan, 'Linux iptables Pocket Reference,' O'Reilly Media, Sebastopol, CA, USA, 2004.
- [11] M. Grinberg, 'Flask Web Development: Developing Web Applications with Python,' 2nd ed., O'Reilly Media, Sebastopol, CA, USA, 2018.
- [12] S. F. Yurkovich, 'Next-Generation Firewalls: A Comprehensive Guide,' Cisco Press, Indianapolis, IN, USA, 2021.

APPENDIX A: COMPLETE SOURCE CODE LISTING

The complete source code for the application is maintained in the GitHub repository. The following excerpt shows the core risk scoring and approval logic from app.py:

```
def calculate_risk(source_ip, destination_ip, port, protocol,
action):
    score = 0
    very_dangerous = [22, 23, 445, 3389]
    dangerous_ports = [22, 23, 25, 53, 110, 143, 445, 3389, 5432,
3306]
    if port in very_dangerous:
        score += 4
    elif port in dangerous_ports:
        score += 2
    elif port < 1024:
        score += 1
    if source_ip == '0.0.0.0/0' or source_ip.lower() == 'any':
        score += 3
    elif source_ip.startswith(('10.', '192.168.', '172.')):
        score += 0
    else:
        score += 2
    if protocol.upper() == 'UDP':
        score += 1
    if action.upper() == 'ALLOW':
        score += 1
    elif action.upper() in ['DROP', 'DENY']:
        score -= 2
    score = max(0, min(10, score))
    if score <= 3: level = 'LOW'
    elif score <= 6: level = 'MEDIUM'
    else: level = 'HIGH'
    return score, level

@app.route('/approve/<int:req_id>', methods=['POST'])
def approve_request(req_id):
    if 'user_id' not in session:
        return redirect(url_for('login'))
    if session['role'] not in ['security_officer', 'admin']:
        flash("Permission denied.")
        return redirect(url_for('dashboard'))
    conn = get_db()
    req = conn.execute("SELECT * FROM requests WHERE id = ?",
(req_id,)).fetchone()
    if not req:
        flash("Request not found.")
        return redirect(url_for('approve_list'))
    if req['requester_id'] == session['user_id']:
        flash("Four-Eyes Principle: You cannot approve your own
request!")
        return redirect(url_for('approve_list'))
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

```

        conn.execute('''UPDATE requests SET status = 'approved',
approved_by = ?,
        approved_at = ?, implemented_at = ? WHERE id = ?''',
        (session['user_id'], now, now, req_id))
    conn.commit()
    conn.close()
    add_audit_entry(req_id, 'APPROVED_AND_IMPLEMENTED',
                    session['user_id'], session['full_name'],
                    f"Request approved by {session['full_name']}.")
    flash(f"Request #{req_id} approved and implemented
successfully!")
    return redirect(url_for('approve_list'))

```

The full application code, HTML templates, and configuration files are available in the GitHub repository referenced on the cover page.

APPENDIX B: DATABASE SCHEMA AND ENTITY RELATIONSHIP

The database consists of three tables with the following relationships. The users table is referenced by requests (requester_id, approved_by, rejected_by, rolled_back_by) and by audit_log (performed_by). The requests table is referenced by audit_log (request_id). All foreign key relationships are enforced at the application level through explicit SQL joins.

Entity Relationships:

```
users ||--o{ requests : submits
users ||--o{ requests : approves
users ||--o{ requests : rejects
users ||--o{ requests : rollbacks
users ||--o{ audit_log : performs
requests ||--o{ audit_log : generates
```

```
users (id PK, username, password_hash, role, full_name)
requests (id PK, requester_id FK, title, source_ip, destination_ip,
          port, protocol, action, reason, risk_score, risk_level,
          status, firewall_command, created_at, approved_by FK,
          approved_at, rejected_by FK, rejected_at, rejection_reason,
          implemented_at, rolled_back, rolled_back_at,
          rolled_back_by FK)
audit_log (id PK, request_id FK, action, performed_by FK,
           performed_by_name, timestamp, details)
```

The use of performed_by_name in audit_log is intentional. Even if a user account is later renamed or deleted, the audit trail retains the human-readable name that was current at the time of the action. This ensures historical accuracy for compliance investigations.